

TEMA N° 2



TEMA 2 LENGUAJE DE PROGRAMACIÓN E IDES



EN ESTE TEMA SERÁS CAPAZ DE...

1. Comprender las fases fundamentales del desarrollo de software y clasificar los tipos de lenguajes de programación para seleccionar la herramienta adecuada ante un problema real.
2. Dominar el uso de un Entorno de Desarrollo Integrado (IDE), desde su instalación hasta la codificación, ejecución y recuperación de código fuente.
3. Escribir y estructurar correctamente la sintaxis básica de un programa informático, aplicando buenas prácticas de desarrollo desde el primer día.



PRÁCTICA



¿Alguna vez te has preguntado cómo las aplicaciones que usas todos los días pasaron de ser una simple idea en la mente de alguien a una herramienta funcional en tu celular o computadora?

Imagina la siguiente situación en nuestra comunidad de Pailón: En el Barrio 27 de Mayo, el coliseo municipal está organizando el campeonato interbarrios de Futsal más grande del año. Participarán equipos del Barrio Central Fátima, Barrio 24 de Septiembre, Residencial Norte, María Belén, San Antonio, Jardines de Pailón, Oto Waver, Ovidio Barberi, San Expedito, Las Misiones y San José. Actualmente, el comité organizador lleva el registro de los equipos, los puntos, los goles y las tarjetas amarillas en un cuaderno escolar que ya tiene las hojas sueltas y manchadas. La información tarda días en actualizarse y los capitanes de los equipos de los barrios más alejados deben caminar hasta el coliseo solo para ver la tabla de posiciones pegada en la pared.

Tú, como futuro Técnico en Sistemas Informáticos formándote en el nuevo módulo educativo del CEA Herman Ortiz Camargo en el Barrio Saó, ves una oportunidad brillante. Sabes que con las computadoras del nuevo laboratorio podrías crear un sistema donde se registren los equipos y la tabla de posiciones se calcule sola. Pero, ¿por dónde empiezas? ¿Cómo le das instrucciones a la computadora para que entienda que si un equipo de San José gana, debe sumar 3 puntos? ¿Dónde escribes esas instrucciones? ¿Qué herramientas necesitas instalar en la computadora del CEA para empezar a construir esta solución?

Piensa en los pasos lógicos que tendrías que seguir antes de tocar el teclado.



TEORÍA



2.1. FASES DE LA PROGRAMACIÓN Y TIPOS DE LENGUAJES

Para que un Técnico en Sistemas Informáticos pueda resolver el problema del campeonato en el Barrio 27 de Mayo, no puede simplemente sentarse frente a la computadora y empezar a teclear al azar. El desarrollo de software es un proceso ordenado que se divide en fases esenciales:



1. **Análisis del problema:** Entender qué se necesita. En nuestro caso, llevar el control de equipos, partidos y puntajes.
2. **Diseño (Algoritmo):** Planificar la solución paso a paso. Es como hacer el plano de una casa antes de poner los ladrillos.
3. **Codificación:** Traducir ese diseño a un **lenguaje de programación** que la computadora pueda procesar.
4. **Pruebas (Testing):** Verificar que el programa no tenga errores (ej. que no asigne 4 puntos por victoria).
5. **Mantenimiento:** Mejorar el programa con el tiempo o adaptarlo a nuevas reglas del campeonato.

Para la fase de codificación, utilizamos los **lenguajes de programación**, que son sistemas de comunicación con reglas gramaticales y sintácticas propias. Se dividen principalmente según su nivel de abstracción (qué tan cerca están del lenguaje humano):

1. **Lenguajes de Bajo Nivel:** Se acercan mucho al "idioma" de la máquina (ceros y unos). Son muy rápidos pero difícilísimos de leer para un humano. Ejemplo: Lenguaje Ensamblador (Assembly).
2. **Lenguajes de Alto Nivel:** Utilizan palabras en inglés (como if, else, print) que son muy fáciles de entender para nosotros. Un Técnico suele trabajar casi exclusivamente con estos lenguajes. Ejemplos: Python, JavaScript, Java, C++.



2.2. DIFERENCIA ENTRE PROCESO DE COMPILACIÓN Y EJECUCIÓN

Una vez que hemos escrito nuestro código en un lenguaje de alto nivel (lo que llamamos **código fuente**), tenemos un problema: la computadora, en su núcleo (el procesador), solo entiende código máquina (impulsos eléctricos representados por 0s y 1s). Por lo tanto, necesitamos un "traductor". Aquí entran dos conceptos vitales: la compilación y la interpretación, previos a la ejecución.

1. **Proceso de Compilación:** Imagina que un vecino del Barrio Central Fátima escribe un libro entero en español y quiere que alguien que solo habla inglés lo lea. El compilador actúa como un traductor que toma **todo el libro completo**, lo traduce entero de una sola vez, y entrega un nuevo libro en inglés. En programación, el compilador toma todo tu código fuente y crea un archivo ejecutable (como un .exe en Windows). Si hay un solo error en el código, el compilador se detiene y no genera el archivo final. Lenguajes compilados: C, C++.
2. **Proceso de Interpretación:** Es como tener un intérprete en vivo. El traductor lee la primera frase, la traduce y se ejecuta; luego la segunda, y así sucesivamente. Si hay un error en la línea 10, el programa funciona hasta la línea 9 y luego "explota" o se detiene. Lenguajes interpretados: Python, JavaScript.



3. **Ejecución:** Es el momento en el que el procesador de la computadora recibe las instrucciones ya traducidas (ya sea por un compilador o un intérprete) y realiza las acciones, como mostrar la tabla de posiciones en la pantalla del laboratorio del CEA.

2.3. DESCRIPCIÓN DEL ENTORNO DE DESARROLLO (IDE) E INSTALACIÓN

Si un mecánico del Barrio Ovidio Barberi necesita reparar el motor de una vagoneta, no usa solo un destornillador suelto; utiliza un taller completo equipado con llaves, gatos hidráulicos, compresores y manuales. Para un Técnico en Sistemas Informáticos, ese taller es el **IDE (Integrated Development Environment** o Entorno de Desarrollo Integrado).

Un IDE es un programa de computadora que agrupa todas las herramientas necesarias para escribir software en una sola interfaz. Componentes principales de un IDE:

1. **Editor de código:** Un bloc de notas hipervitaminado que resalta las palabras clave con colores para que no te equivoques.
2. **Compilador/Intérprete:** La herramienta integrada para traducir tu código.
3. **Depurador (Debugger):** Una lupa que te permite pausar la ejecución del código línea por línea para encontrar errores.

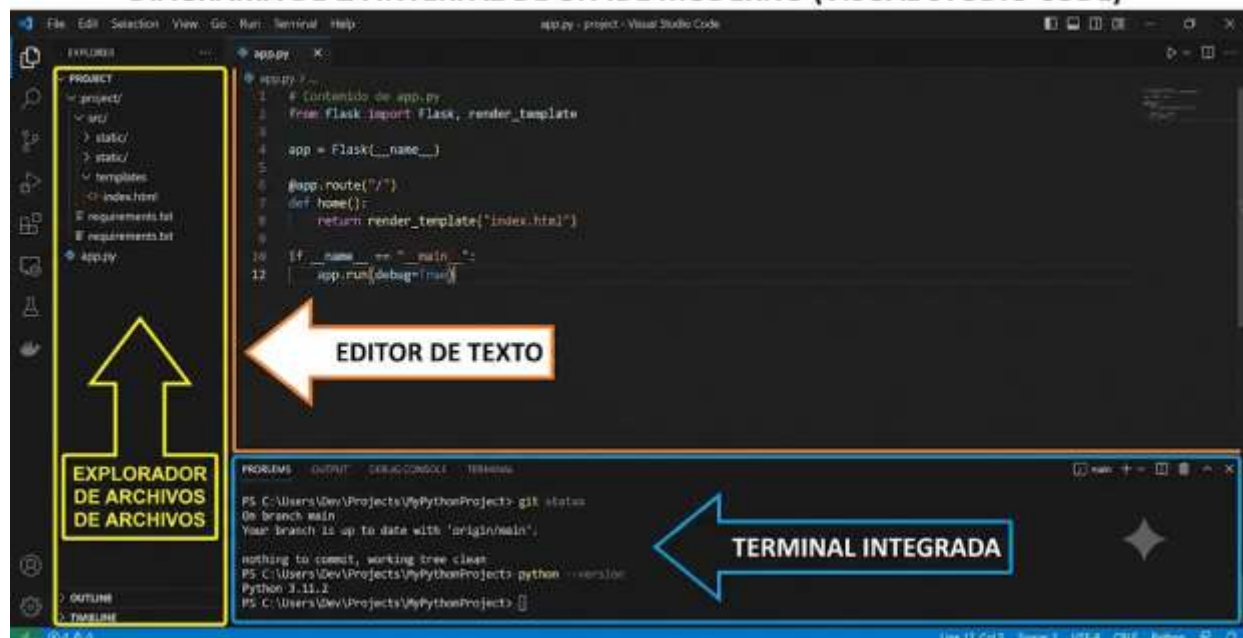
Instalación recomendada (Visual Studio Code):

Para el laboratorio del CEA Herman Ortiz Camargo, la herramienta estándar de la industria es Visual Studio Code (VS Code).

1. Ingresar a code.visualstudio.com desde el navegador.
2. Descargar el instalador para Windows, macOS o Linux según el sistema del CEA.
3. Ejecutar el instalador dando clic en "Siguiente", aceptando los términos y asegurándose de marcar la opción "Agregar al PATH" (esto permite que la computadora reconozca los comandos del IDE).
4. Instalar "Extensiones" clave desde el panel lateral izquierdo, como el paquete de idioma Español y las extensiones del lenguaje que vayamos a usar (ej. Python).



DIAGRAMA DE LA INTERFAZ DE UN IDE MODERNO (VISUAL STUDIO CODE)



2.4. ESTRUCTURA SINTÁCTICA BÁSICA DE UN PROGRAMA

Todo lenguaje de programación tiene una estructura y reglas ortográficas estrictas, conocidas como **sintaxis**. Si escribes mal una palabra o te falta un símbolo, la computadora simplemente no te entenderá.

Analicemos la estructura básica escribiendo un pequeño código en Python que registra un equipo del campeonato en Pailón. Observa cómo el código sigue un flujo lógico, desde la recolección de datos hasta el procesamiento y la salida de información.

Python

```
# Esto es un comentario. La computadora lo ignora. Sirve para que el Técnico entienda el código.
# Paso 1: Definición de variables (espacios en la memoria para guardar datos)
nombre_equipo = "
Los Halcones del Barrio 24 de Septiembre" # Guardamos texto entre comillas
puntos_ganados = 0 # Guardamos un número entero inicial
# Paso 2: Entrada de datos (solicitamos información al usuario)
partidos_ganados = int(input("
Ingrese la cantidad de partidos ganados: ")) # Convertimos el texto ingresado a número entero
# Paso 3: Procesamiento lógico y matemático
# En el Fútbol, cada partido ganado suma 3 puntos
puntos_ganados = partidos_ganados * 3 # Multiplicamos usando el asterisco
```



```
# Paso 4: Salida de información (mostramos el resultado en pantalla)
print("
El equipo", nombre_equipo) # Imprime el texto y la variable
print("
Tiene un total de puntos de:", puntos_ganados) # Imprime el cálculo final
```

Como observas en el bloque anterior, un programa básico siempre tiene: Variables para guardar información, Entradas para recibir datos, Procesos para manipularlos y Salidas para mostrar el resultado.

2.5. CODIFICACIÓN, GUARDADO Y RECUPERACIÓN DE CÓDIGO FUENTE

El código no puede quedarse flotando en la memoria RAM, porque si se corta la luz en el Barrio Saó, perderás horas de trabajo.

1. **Codificación:** Es el acto físico de escribir las instrucciones en el IDE.
2. **Guardado:** Los archivos de código fuente son, en esencia, archivos de texto plano, pero se guardan con una extensión específica según el lenguaje. Por ejemplo, si programas en Python, guardas tu archivo como campeonato.py. Si programas en JavaScript, como logica.js. Al guardarlo en el disco duro de la computadora del CEA, aseguras su persistencia.
3. **Recuperación:** Al día siguiente, abres tu IDE (VS Code), vas a la opción "Archivo" > "Abrir carpeta" (File > Open Folder) y seleccionas la carpeta donde guardaste tu proyecto. El IDE automáticamente cargará tus archivos .py o .js listos para continuar editando.

2.6. ASISTENTES DE INTELIGENCIA ARTIFICIAL EN LA ESCRITURA DE CÓDIGO

Actualmente, un Técnico no trabaja solo. La Inteligencia Artificial (IA) ha revolucionado cómo escribimos código. Herramientas como GitHub Copilot, ChatGPT o Claude pueden integrarse directamente en tu IDE o usarse en el navegador.

Estas IAs funcionan como un "compañero de programación experto". Si no recuerdas cómo hacer un cálculo matemático complejo para el campeonato de Pailón, puedes escribir un comentario en tu código como `# Función para calcular promedios de gol` y la IA te sugerirá el código completo. Es crucial entender que **la IA no reemplaza al programador**; solo le da velocidad. El Técnico es quien debe revisar, entender y aprobar el código sugerido, ya que las IAs pueden cometer errores o proponer soluciones ineficientes.



2.7. CONTROL DE VERSIONES EN LA NUBE (GIT Y GITHUB) PARA EL TRABAJO COLABORATIVO

Imagina que dos estudiantes del CEA están haciendo el sistema del coliseo juntos. Uno programa la pantalla de registro y el otro programa la tabla de posiciones. Si se pasan los archivos por pendrive (flash memory), terminarán con archivos como proyecto_final.py, proyecto_final_ahorasi.py, proyecto_final_el_verdadero.py. Esto es un desastre organizativo.

Para evitar esto, en la actualidad se utilizan sistemas de control de versiones como **Git** apoyados en plataformas en la nube como **GitHub**. Esto permite que tu código fuente se guarde de forma segura en internet. Cada vez que haces un cambio importante, guardas una "fotografía" (commit) de tu código. Si te equivocas y arruinas el programa, puedes "viajar en el tiempo" a una versión anterior. Además, permite que varios Técnicos trabajen en el mismo proyecto al mismo tiempo sin borrar el código del otro.

2.8. ENTORNOS DE DESARROLLO BASADOS EN LA NUBE (CLOUD IDES)

¿Qué pasa si un día no puedes ir al módulo del CEA y la computadora que tienes en tu casa en el Barrio San Exposito es muy antigua o no tiene espacio para instalar Visual Studio Code? La tecnología actual ofrece los **Cloud IDEs** (IDEs en la nube).



Plataformas como GitHub Codespaces o Replit te permiten abrir un entorno de desarrollo completo, idéntico a VS Code, directamente dentro de tu navegador web (Google Chrome o Firefox). Todo el procesamiento, la compilación y la ejecución ocurren en servidores potentes en internet. Solo necesitas una conexión a internet, lo que democratiza el acceso a la programación para cualquier Técnico, sin importar la potencia de su equipo local.

CIERRE TEÓRICO OBLIGATORIO

Mito Común	Realidad Técnica
"Se necesita saber matemáticas avanzadas y ser un genio para programar."	FALSO. La mayoría de la programación requiere lógica básica, orden y atención al detalle. Saber sumar, restar, multiplicar y dividir es suficiente para el 80% del trabajo inicial.
"Escribir código más rápido y en menos líneas siempre hace un mejor programa."	FALSO. El mejor código es aquel que es fácil de leer y entender por otro Técnico o por ti mismo meses después. La claridad es superior a la brevedad.

□ Glosario de Términos:

1. **Código Fuente:** Archivo de texto que contiene las instrucciones escritas en un lenguaje de programación comprensible para el ser humano.
2. **Sintaxis:** Conjunto de reglas ortográficas y gramaticales que deben seguirse obligatoriamente al escribir en un lenguaje de programación.
3. **Compilador:** Herramienta que traduce el código fuente completo a código máquina antes de su ejecución.
4. **IDE (Entorno de Desarrollo Integrado):** Aplicación de software que proporciona facilidades completas (editor, depurador, terminal) a los programadores para el desarrollo de software.
5. **Variable:** Espacio reservado en la memoria de la computadora que sirve para almacenar datos temporales durante la ejecución de un programa.

**Ponte a Prueba:**

1. Si cometo un error ortográfico al escribir un comando, ¿qué parte del lenguaje estoy violando?
 - a) La compilación.
2.
 - b) La sintaxis.
3.
 - c) La ejecución.
4. ¿Cuál de las siguientes herramientas actúa como un taller completo para el Técnico?
 - a) Un navegador web.
5.
 - b) El Bloc de Notas.
6.
 - c) Un IDE (como VS Code).
7. ¿Qué lenguaje requiere traducir todo el código de una sola vez para crear un archivo ejecutable?
 - a) Lenguaje Compilado.
8.
 - b) Lenguaje Interpretado.
9.
 - c) Lenguaje de Bajo Nivel únicamente.

(Respuestas: 1-b, 2-c, 3-a)

**VALORACIÓN**

Aprender a programar nos otorga un superpoder digital, pero conlleva una gran responsabilidad que como Técnicos en Sistemas Informáticos debemos analizar:

1. **Impacto Ambiental y E-waste:** El nuevo módulo del CEA Herman Ortiz Camargo está equipado con computadoras modernas. Pero, ¿qué pasa con los equipos viejos que se descartan en Pailón? Como programadores, escribir código eficiente (que no consuma demasiada memoria o procesador) ayuda a que las computadoras antiguas sigan siendo útiles por más tiempo, reduciendo la basura electrónica (e-waste). Además, los servidores



en la nube consumen enormes cantidades de electricidad. Software optimizado significa menor impacto de huella de carbono.

2. **Responsabilidad Social y Local:** Si creamos el sistema para el coliseo del Barrio 27 de Mayo, estamos aportando al desarrollo tecnológico de Pailón, demostrando que no necesitamos comprar software extranjero, sino que el talento local puede resolver problemas locales.
3. **Privacidad y Seguridad de la Información:** Al manejar datos de los vecinos (nombres, edades, direcciones de los jugadores de Las Misiones o San Antonio), tenemos la obligación ética de aplicar seguridad. Escribir buen código asegura que la información de nuestra comunidad no quede expuesta a robos de identidad o alteraciones maliciosas.



PRODUCCIÓN



RETO FINAL: "TU PRIMER PROGRAMA EN PAILÓN"

Ha llegado el momento de aplicar la teoría. Tu misión es instalar tu entorno de trabajo en la computadora del laboratorio del CEA y crear tu primer archivo ejecutable que registre datos de prueba para el torneo interbarrios.

Paso 1: Preparación del Entorno (IDE)

1. Enciende tu equipo en el laboratorio del CEA.
2. Busca e inicia **Visual Studio Code**. (Si no está instalado, descárgalo de su sitio oficial).
3. Ve a la sección de "Extensiones" (cuadritos en el menú izquierdo) e instala la extensión de "Python" (oficial de Microsoft).

Paso 2: Creación del Espacio de Trabajo

1. En el escritorio de tu computadora, crea una carpeta nueva llamada Torneo_Pailon.
2. Arrastra esa carpeta adentro de la ventana de Visual Studio Code para abrirla como tu espacio de trabajo.



- Dentro de VS Code, crea un nuevo archivo dando clic al ícono de "Nuevo archivo" y llámalo registro.py. Asegúrate de escribir bien la extensión .py.

Paso 3: Codificación (Escribiendo la Sintaxis)

Copia cuidadosamente el siguiente código en tu archivo `registro.py`. Fíjate bien en la sintaxis (paréntesis, comillas y mayúsculas).

Python

[illegible]



```
---  
---  
--")  
# Solicitamos datos por teclado nombre_barrio = input("  
Ingrese el nombre de su barrio (Ej. Saó, Fátima): ")  
nombre_capitan = input("  
Ingrese el nombre del capitán del equipo: ")  
# Mostramos el registro exitosoprint("\n  
--- RESUMEN DEL REGISTRO -  
--")  
print("  
El equipo del barrio", nombre_barrio, "  
ha sido inscrito exitosamente.")  
print("  
Capitán a cargo:", nombre_capitan)  
print(";El CEA Herman Ortiz Camargo les desea mucho éxito!")
```

Paso 4: Guardado y Ejecución

1. Guarda tu progreso usando el atajo de teclado Ctrl + S (o ve a Archivo > Guardar).
2. Para ejecutarlo (interpretarlo), en VS Code ve al menú superior y haz clic en "Terminal" > "Nueva Terminal".
3. En la consola negra que se abre abajo, escribe el siguiente comando y presiona la tecla Enter:

Bash

```
# Comando para ejecutar el intérprete de Python sobre nuestro archivopython  
registro.py
```

1. Interactúa con tu programa, responde las preguntas y verifica que los datos de salida sean correctos. ¡Felicidades! Has programado tu primer sistema operativo funcional adaptado a la realidad de Pailón.

RECURSOS ADICIONALES

Para profundizar en el dominio de los lenguajes de programación y tu IDE, como Técnico te sugiero explorar los siguientes recursos oficiales:

1. **Documentación Oficial de Visual Studio Code:** Excelente para aprender atajos de teclado y trucos de configuración del IDE que te ahorrarán horas de trabajo. (<https://code.visualstudio.com/docs>)



2. **MDN Web Docs (Mozilla Developer Network):** La biblioteca más completa si decides orientarte a lenguajes web como JavaScript. Ofrece tutoriales paso a paso y referencias sintácticas impecables. (<https://developer.mozilla.org/es/>)
3. **Python.org (Tutorial para Principiantes):** Documentación oficial del lenguaje Python, ideal para revisar en detalle la sintaxis básica, tipos de datos y funciones avanzadas en español. (<https://docs.python.org/es/3/tutorial/index.html>)